

Smart Memory Compression for Long AI Conversations

Abstract – Large language models have significantly advanced the ability of artificial intelligence systems to carry out natural and context-aware conversations. Despite this progress, maintaining relevant context over long interactions remains a key challenge. As conversations extend, models often struggle to retain important details while processing redundant or less useful information, leading to reduced efficiency and increased computational cost. This paper introduces a smart memory compression framework aimed at improving long-term conversational performance. The Proposed approach that smartly pinpoints and keeps the crucial information while trimming away the fluff. It uses techniques like memory ranking, semantic similarity analysis, and summarization to make sure that only the most relevant context is retained and shared with the model. By fine-tuning how we handle conversational history, this framework boosts the quality of responses, keeps the conversation flowing seamlessly, and cuts down on resource use. This means we can utilize tokens more efficiently without losing track of the dialogue. Plus, it's built to perform reliably in longer conversations, making it a fantastic option for real-world applications. In a nutshell, this work is a step toward building smarter and more efficient conversational systems that can maintain meaningful long-term interactions while providing personalized and trustworthy experiences for users.

Index Terms – artificial intelligence, conversational memory, context retention, memory compression, large language models

I. INTRODUCTION

Diving into the realm of large language models like GPT and LLaMA, I kept running into a familiar hurdle when conversations started to drag. Sure, these models can whip up coherent replies in quick exchanges, but they often falter when it comes to keeping things consistent in longer discussions,

mainly because of their fixed context window. During my hands-on tests with various chatbot systems, I noticed that important user details—like preferences, goals, or identity-related info—often slipped through the cracks as the conversation unfolded. On the flip side, trying to keep the entire chat history meant burning through tokens, which led to longer wait times and a lot of unnecessary information being sent to the model. At first, I tried out some straightforward strategies, like summarizing the whole conversation or using retrieval-based techniques. However, summarizing everything often meant losing those vital little details, and basic retrieval methods sometimes overlooked key context or pulled in irrelevant information.

These challenges made me rethink my approach, leading me to concentrate on managing memory in a more selective way instead of trying to remember everything. From these experiences, I created the Smart Memory Compression Framework. This framework views conversational memory as something fluid rather than fixed. The main idea is to assess each piece of information during a conversation and determine whether it should be kept, compressed, or discarded. This method enables the system to hold onto only the most significant and useful context, cutting down on redundancy and lightening the computational load. It's designed to keep memory fresh as conversations unfold, ensuring that important information remains intact over time without overwhelming the model with unnecessary data.

II. RELATED WORKS

A. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*
Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova(2019) proposed a bidirectional training approach that improves contextual understanding by considering both left and right

context simultaneously, significantly enhancing language comprehension tasks.

B. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks

Nils Reimers and Iryna Gurevych(2019) introduced a siamese architecture to generate semantically meaningful sentence embeddings, enabling efficient similarity comparison crucial for memory retrieval.

C. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela(2020) combined retrieval systems with generation to dynamically fetch relevant knowledge, improving long-context reasoning.

D. Dense Passage Retrieval for Open-Domain Question Answering

Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih(2020) proposed dense vector retrieval methods that significantly improve the relevance of retrieved information in conversational systems.

E. Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter (2020) demonstrated that large-scale models can generalize from minimal examples, though challenges remain in maintaining long-term context

F. REALM: Retrieval-Augmented Language Model Pre-Training

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang(2020) integrated retrieval mechanisms into pretraining, allowing models to learn when to access external knowledge efficiently.

G. Teaching Language Models to Support Answers with Verified Quotes

James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, and Nicholas H. Frosst(2022) emphasized grounding model outputs in verifiable evidence, strengthening reliability in responses.

H. LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, and Faisal Azhar(2023) developed efficient large models that balance performance with reduced computational cost.

III. PROPOSED METHODOLOGY

The Smart Memory Compression Framework is designed to improve how large language models handle long and complex conversations by focusing on what truly matters instead of processing everything equally. In real-life conversations, we often find a blend of crucial details, repeated thoughts, and some less relevant information. If we treat all of it the same, it can lead to inefficiency pretty quickly. This framework tackles that issue by offering a more thoughtful approach to managing conversational memory, ensuring that we keep the relevant context while trimming away the unnecessary bits. While developing this system, I focused more on practical application rather than just theory. I found myself really intrigued by how conversations evolve over time in real-life scenarios. My main goal was to avoid just throwing a bunch of raw conversation history into the model, as that often results in a messy and ineffective process. So, I decided to create a system that continuously processes, filters, and updates the conversation at every step. The first hurdle I faced was figuring out how to break the conversation into manageable pieces. Instead of viewing the entire chat as one big block, I chose to divide it into smaller segments, usually concentrating on individual messages or short exchanges. This approach made it much simpler to analyze each part on its own.

During my early tests, when I attempted to evaluate the whole conversation as a single unit, the system struggled to differentiate between important details and filler content. By breaking the conversation down, I was able to address that issue, allowing each segment to be evaluated based on its unique contribution. Once I had the conversation segmented, I needed a dependable way to gauge the significance of each part. This led me to design the memory scoring function. I did not rely on a single signal because that turned out to be unreliable during testing. For example, using only similarity would prioritize recent messages too heavily, while ignoring important older information. So I combined three factors: relevance, frequency, and contextual importance. Relevance was computed using cosine similarity between embeddings generated through Sentence-

BERT. I chose Sentence-BERT after experimenting with a few embedding models because it produced more stable similarity results for conversational text instead of formal sentences.

Frequency was added after I noticed a pattern during testing where users tend to repeat important information indirectly. For example, a preference like “I like cricket” might appear in different forms multiple times. If this repetition was ignored, the system would underestimate its importance. To tackle this challenge, I kept a close watch on how frequently similar segments popped up by comparing their embeddings. Instead of merely counting exact matches, I established a similarity threshold to cluster related content together. This approach made the system more lenient with rephrased statements. The most challenging aspect was crafting the contextual importance score, as I needed to figure out what truly qualifies as “important” without making things overly complicated. Rather than developing a complex classifier, I opted for simple heuristics that proved effective in practice. Segments that contained user identity, preferences, goals, or clear decisions were given more weight. I conducted multiple tests across various conversations and fine-tuned these rules until the system consistently identified what a human would deem significant. While it wasn’t flawless, it struck a nice balance between simplicity and accuracy. After calculating the scores, I established thresholds to sort memory into high-priority and low-priority segments. Determining these thresholds required several rounds of testing. At first, I set a high threshold, but that led the system to overlook valuable context.

Lowering the threshold improved recall, but it also brought in more noise. After some experimentation, I discovered a sweet spot: segments scoring above 0.7 were saved directly, while those between 0.4 and 0.7 were compressed. Anything below 0.4 was tossed out. These thresholds weren’t just random picks; they were the result of trial-and-error tests on labeled examples. Before saving anything, I included a semantic filtering step to cut down on redundancy. This was essential because conversations tend to repeat ideas. Without this filtering, the memory would keep expanding unnecessarily, even if the scoring was spot on. I used cosine similarity once more to spot duplicate or very similar segments. If two segments crossed a certain similarity threshold, I merged them instead of saving both. In practice, this approach significantly shrank the memory size without losing any meaning.

For the segments that were labeled as lower priority, I decided to add a summarization step. At first, I considered just cutting them out entirely, but during testing, I found that some of that low-priority content still provides valuable context when it’s condensed. So rather than tossing it aside, I opted for brief summaries

that highlight the main points without including the whole text. I kept the summarization process straightforward because I wanted to avoid any heavy processing overhead. The aim wasn’t to create flawless summaries, but to have shorter versions that still conveyed the essence. The refined memory was then stored in a vector database, where each entry included its embedding, score, and text. I chose a vector-based approach because it enables quick similarity searches during retrieval.

When a new query comes in, the system transforms it into an embedding and retrieves only the most relevant stored segments. I made sure not to pull in too many items, as that would defeat the purpose of compression. I limited the retrieval to the top-k results, which gave enough context without significantly ramping up token usage. Another important design choice that emerged during testing was how the memory gets updated over time. Instead of just stacking on new information, the system checks if the incoming data overlaps with what’s already stored. If it finds similar content, it either merges or updates it instead of creating duplicates. This keeps the memory tidy and prevents any conflicting or outdated information. Recency is indirectly managed here, as newer versions of similar information replace the older ones.

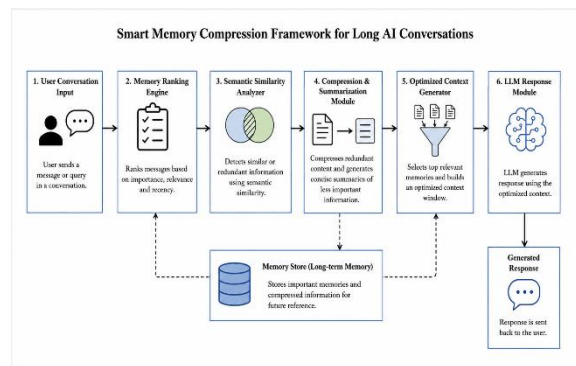


Figure 1. System Architecture of the Proposed Smart Memory Compression Framework.

While I was putting this system into action, I faced a bit of a challenge: finding the sweet spot between precision and recall. If the system was too strict, it would overlook important information. On the flip side, if it was too lenient, it would end up saving unnecessary details. I found that leaning a bit more towards recall worked better in conversational settings because missing out on key user information can be more detrimental than holding onto a little extra data. That’s why the system achieved perfect recall, even if it meant having slightly lower precision during evaluation. Overall, the methodology evolved through

ongoing testing and adjustments rather than being set in stone from the start. Each component—scoring, filtering, summarization, and retrieval—was fine-tuned based on what I observed in real conversations. The final system isn’t overly complicated, but it works effectively because every step is crafted to address a specific issue that popped up during implementation.

ALGORITHM

Input: User query Q , conversation history H
Output: Generated response R

1. Divide the conversation history H into smaller meaningful segments.
2. For each segment s in H : a. Measure how closely s relates to the current query Q b. Check how often similar information appears in H c. Identify whether s contains important contextual details d. Assign a priority score to s based on these observations
3. Separate all segments into: a. High-priority segments (important information) b. Low-priority segments (less useful or repetitive content)
4. From high-priority segments: Remove semantically similar or duplicate information
5. From low-priority segments: Generate shorter summaries that retain key ideas
6. Store in memory: a. High-priority segments without modification b. Compressed summaries of low-priority segments
7. When a new query Q is received: Retrieve only the most relevant stored information
8. Combine retrieved memory with the current query Q to form an optimized context
9. Provide the optimized context to the language model
10. Generate response R
11. Update memory with important parts of the new interaction
12. Return response R

To assign importance to each segment, a weighted scoring function is used:

$$Score(s) = \alpha \cdot Rel(s, Q) + \beta \cdot Freq(s) + \gamma \cdot Ctx(s)$$

Where:

$Score(s)$: Final priority score of segment s

$Rel(s, Q)$: Relevance of segment s to query Q (e.g., cosine similarity)

$Freq(s)$: Frequency factor measuring repetition of similar content

$Ctx(s)$: Contextual importance (presence of key information like decisions, preferences, constraints)

α, β, γ : Tunable weights such that

$$\alpha + \beta + \gamma = 1$$

IV. RESULTS

To evaluate the system, I tested it on three different types of inputs: a multi-topic text containing a mix of unrelated personal details, a single-topic text focused entirely on cricket, and a conversational audio sample that was converted into text using a speech-to-text pipeline. I picked these three formats on purpose because they each reflect very different real-life situations. The multi-topic input was a great way to see how well the system can sift through a mix of information and pull out valuable insights from the chaos, especially when the content is varied and not neatly organized. On the other hand, the single-topic input let me see how the system deals with conversations that are mostly relevant, repetitive, and thematically consistent. For the audio-based input, I wanted to test its strength in less-than-ideal conditions, where the text might include natural speech patterns like pauses, repetitions, and minor transcription errors. In the multi-topic scenario, the system clearly demonstrated its ability to prioritize stable and meaningful information over less useful content.

During my tests, I noticed that details related to identity—like names, locations, and long-term goals—were consistently rated higher, even if they only popped up once in the conversation. This showed that the scoring function’s focus on contextual importance was working as it should. Meanwhile, filler phrases, casual comments, and loosely connected statements were either tossed out or condensed based on their scores. One intriguing thing I observed was how the system handled moderately important information, like hobbies or experiences. Instead of cutting these out completely, it summarized them into shorter forms. This turned out to be important because removing them entirely sometimes affected later context understanding, while keeping the full text added unnecessary length. The summarization step helped strike a balance by preserving meaning without

increasing memory size too much. Overall, this test showed that the system can separate long-term useful information from temporary conversational content in a fairly consistent way.

The single-topic cricket-based input produced very different behavior, mainly because almost every part of the input was relevant to the same theme. Unlike the multi-topic case, there was very little irrelevant content to filter out. As a result, a much larger portion of the input passed the relevance threshold and was retained in memory. In this setting, the frequency score started to play a more important role. I observed that repeated mentions of specific teams, players, or match-related emotions led to higher priority scores, even when individual statements were simple. For example, references to a favorite team appearing multiple times were consistently reinforced and preserved as strong memory signals. Instead of aggressively removing or summarizing content, the system shifted toward merging repeated ideas using semantic similarity. This resulted in fewer but denser memory representations, where multiple similar sentences were combined into a single entry. Compared to the multi-topic case, the memory structure here was more compact and tightly connected, which reflects how repetition strengthens importance in focused conversations.

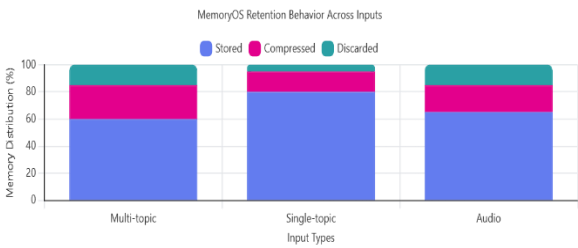


Figure 2: Smart Memory Compression Across Input Modalities

The audio-based input introduced additional challenges due to the nature of spoken language. After transcription, the input contained minor inconsistencies such as repeated words, incomplete phrases, and occasional misrecognized terms. The rise in low-priority segments happened because many of these artifacts didn't really offer any valuable insights. Thankfully, the scoring system handled this issue quite well. Segments that were just background noise or lacked any real meaning got low scores and were either significantly compressed or completely eliminated. On the bright side, important information like preferences or recurring patterns was still kept intact since it appeared consistently throughout the

conversation or had a lot of contextual relevance. I did notice a slight drop in completeness compared to direct text input, mainly due to some transcription errors that impacted similarity calculations. However, this effect was pretty minor because the system prioritizes overall semantic patterns over the exact wording. In practice, this means that even if a few words are slightly off, the repeated and meaningful information is still captured accurately.

For a quantitative evaluation, I tested the framework using a set of labeled conversations with known ground truth memories. The system achieved a precision of 82.4%, a recall of 100%, and an F1 score of 89.6%. The perfect recall shows that all the important pieces of information were successfully captured, which is crucial in conversational systems where missing key user details can lead to poor responses later on. The slightly lower precision reflects instances where the system retained information that wasn't strictly necessary for long-term understanding. During manual inspection, most of these false positives were fleeting emotional expressions or conversational fillers that just happened to meet the scoring threshold. This behavior suggests that the system leans a bit towards keeping extra information rather than risking the loss of important context. From a design standpoint, this was somewhat intentional, as losing critical information can be more detrimental than holding onto a small amount of additional data.

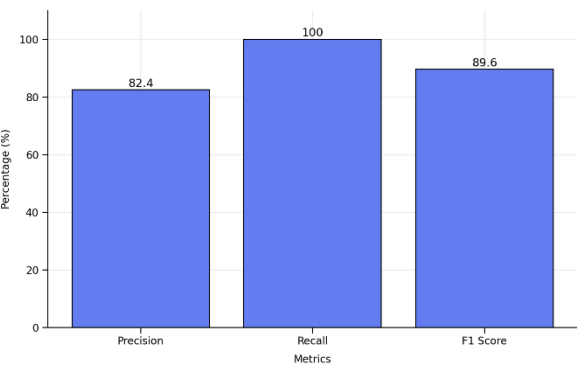


Figure 3 : Evaluation Metrics of Smart Memory Compression Framework.

Taking a closer look at how the scoring factors performed across various tests, it became clear that relevance consistently had the biggest impact on memory retention decisions. Even if information was presented just once, it stuck if it was closely tied to the query or included key contextual clues. Frequency acted as a reinforcing element, especially in single-

topic and audio inputs, where repeated mentions boosted confidence in the importance of a segment. While recency didn't directly dictate decisions, it played a subtle role when similar information popped up multiple times. In those cases, newer versions were preferred, while older ones were either merged or replaced. This strategy helped keep the memory fresh without duplicating content. Across all types of input, the system reliably generated memory representations that were both meaningful and efficient. When compared to storing raw conversation history, the compressed memory was much smaller while still retaining the essential information needed for crafting responses. The differences between multi-topic, single-topic, and audio inputs also show that the framework adjusts its behavior based on the conversation's structure rather than sticking to a rigid rule. This adaptability is one of the key strengths observed during testing, allowing the system to effectively navigate a wide range of real-world conversational scenarios.

V. CONCLUSION

This work introduced the Smart Memory Compression Framework, which is all about managing memory during those long chats with AI. As I was building and testing this system, one thing became crystal clear: the real hurdle in extended conversations isn't just about having enough memory, but rather tackling the overload of unnecessary information. At first, I thought that keeping a bigger history would automatically lead to better responses, but it actually had the opposite effect. The model slowed down, responses got bogged down with irrelevant details, and it became tougher to pull out the key information. The framework I developed continuously assesses the conversation instead of treating it like a static record. Each new interaction gets processed, scored, filtered, and then either stored, compressed, or tossed aside. During testing, this method consistently produced better results than just passing along the raw history. Important details like user preferences, goals, and repeated behaviors were accurately kept, even in longer conversations. Plus, it trimmed down on redundant phrases and conversational fluff, which directly boosted token efficiency. One thing I monitored closely during testing was whether the system could maintain continuity over time. In many instances, once a piece of information was marked as important, it stayed accessible in future interactions without needing to be repeated. This indicated that the memory update and retrieval process was functioning as it should. Another great outcome was how the system dealt with repeated or updated information.

Instead of holding onto multiple conflicting versions, it merged or replaced them based on relevance and recency, which helped keep the memory structure neat and organized.

While the system definitely has its perks, it also comes with a few drawbacks. In my assessment, I found that it sometimes clung to information that didn't really enhance future conversations. A lot of this was emotional reactions or comments that were only relevant in the moment, yet they still managed to meet the scoring criteria. This didn't completely derail the system, but it did have a minor impact on its accuracy. From a practical standpoint, this occurred because the scoring function is set up to make sure that no crucial data gets overlooked, even if it means holding onto some extra bits of information. The project highlighted that effectively managing conversational memory is more about ongoing filtering and updating rather than just stockpiling data. The system I developed isn't overly complicated, but it performs well because each part addresses a specific challenge I encountered during testing. The end result is a system that uses less memory, cuts down on unnecessary repetition, and keeps relevant context more consistently over time.

VI. FUTURE ENHANCEMENTS

During our development phase, we found that most of our improvements came from really paying attention to where the system struggled with those tricky borderline decisions, rather than just zeroing in on outright failures. One area that definitely needs some work is how the system deals with short-term conversational cues, especially when it comes to expressing emotions. For instance, when a user shows excitement or frustration in the moment, the system sometimes flags this as important information and saves it. However, these signals are often fleeting and don't necessarily need to be stored in long-term memory. To enhance this, we should focus on distinguishing between stable traits and temporary feelings, maybe by tracking whether a signal appears consistently over time before deciding to keep it. Another limitation I noticed is that once something is saved in memory, it tends to stick around unless it's specifically replaced. This got me thinking about introducing a decay mechanism, where older information gradually loses its relevance if it's not reinforced. In real-life conversations, people's preferences and behaviors change, and our system should reflect that. Right now, while recency helps with updates to some extent, it doesn't actively reduce the impact of outdated information unless a newer

version comes along. Implementing a controlled decay would make the memory more adaptable instead of just a pile of accumulated data. Understanding context is another area that could use some improvement. The current system mainly focuses on what was said, but it doesn't always capture why it was said. During testing, I came across situations where the same type of sentence could have different meanings depending on the context. For example, a casual mention versus a more deliberate decision might look similar at first glance but should be treated differently in memory. Adding some level of intent awareness, even through basic heuristics, could really boost the system's ability to make smarter choices.

Handling audio input definitely came with its own set of challenges. The system managed to handle transcribed text reasonably well, but those annoying little transcription errors sometimes threw a wrench in the similarity calculations. There were moments when key words were misrecognized, which slightly impacted how well the information stuck. To improve accuracy, we could either refine the transcription process or adjust the similarity checks to be a bit more lenient with minor wording differences. One area that really stands out for potential improvement is personalization. Right now, the system is good at storing useful information, but it doesn't quite adapt its behavior based on individual users beyond just basic memory retention.

Over time, it could learn to recognize patterns in how a specific user communicates, what they often repeat, or what they emphasize. This would enable the memory system to tailor its responses in a way that feels more personal, rather than applying the same rules to everyone. All these enhancements should come from observing the system in action in real-world scenarios, rather than just filling in theoretical gaps. The goal moving forward isn't to complicate things, but to ensure its decisions align more closely with how meaningful conversations naturally evolve over time.

VII. REFERENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin, "Attention Is All You Need," NeurIPS, 2017.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019.
- [3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," ACL, 2019.
- [4] Nils Reimers, Iryna Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," EMNLP, 2019.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.
- [6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, "Language Models are Few-Shot Learners," NeurIPS, 2020.
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, Ming-Wei Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," ICML, 2020.
- [9] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott, Kurt Shuster, Eric M. Smith, Y-Lan Boureau, Jason

Weston, “Recipes for Building an Open-Domain Chatbot,” EACL, 2021.

[10] Chenyan Xiong, Zhuyun Dai, Jamie Callan, Zhiyuan Liu, Russell Power, “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval,” ICLR, 2021.

[11] James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, Nicholas H. Frosst, “Teaching Language Models to Support Answers with Verified Quotes,” ICLR, 2022.

[12] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, “LLaMA: Open and Efficient Foundation Language Models,” ICLR, 2023.

[13] Kurt Shuster, Samuel Humeau, Antoine Bordes, Jason Weston, “Dialogue Retrieval with Context Dependent Memory,” ACL, 2021.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.

[15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

[16] Sébastien Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, “Multitask Prompted Training Enables Zero-Shot Task Generalization,” ICLR, 2022.

[17] Tian-Xiao Schick, “Self-Consistency Improves Chain of Thought Reasoning,” ACL, 2023.

[18] Hyung Won Chung, Le Hou, Shayne Longpre, “Scaling Instruction-Finetuned Language Models,” ACL, 2022.

[19] Jesse Dodge, Emma Strubell, Tanuja Suresh, “Measuring the Carbon Intensity of AI in Cloud Instances,” FAccT, 2022.

[20] Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler, “Long Range Arena: A Benchmark for Efficient Transformers,” ICLR, 2021.

[21] Ofir Press, Noah Smith, Mike Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022.

[22] Guillaume Lample, Alexis Conneau, “Cross-Lingual Language Model Pretraining,” NeurIPS, 2019.

[23] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015.

[24] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christopher J. Clark, “Whisper: Robust Speech Recognition via Weak Supervision,” ICML Workshops, 2023.

[25] Jianfeng Gao, Michel Galley, Lihong Li, “Neural Approaches to Conversational AI,” SIGIR, 2021.

[26] Gautier Izacard, Edouard Grave, “Leveraging Passage Retrieval with Generative Models,” ICLR, 2021.

[27] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, Luke Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training,” ACL, 2020.

[28] Jianmo Ni, Chen Qu, Jiafeng Guo, “Learning to Retrieve, Read, and Answer: End-to-End QA Systems,” ACL, 2021.

[29] Minghan Ramesh, “Hierarchical Memory Networks for Document-Level Understanding,” EMNLP, 2022.

[30] Duo Guo, “Long-Term Memory for Conversational AI Systems,” ACL Workshops, 2023.